



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	Lahari Bhargav
Student Name	Dhatrika Deekshitha
Roll Number	2520030216
Branch	CSE
Course Outcome	CO3

Case Study Topic: ParkSmart – Parking Zone Connectivity(Graph Traversal & Minimum Spanning Trees)

Work Summary:

In this case study, we applied Kruskal's algorithm on a 12-edge candidate graph connecting 6 parking zones {A, B, C, D, E, F} to the Central Parking Control Hub (P). We sorted all road-network connections by cost in ascending order, applied Union-Find to greedily build the Minimum Spanning Tree (MST), and recorded each accept/reject decision with parent-pointer state.

We also analysed the naive Union-Find implementation (without union-by-rank and path compression) and compared its worst-case complexity with the optimised version. Finally, we verified whether the resulting parking network satisfies the redundancy requirement of maintaining at least two edge-disjoint routes between the Central Hub (P) and Parking Zone B.

Since the MST alone does not satisfy this requirement, we proposed the minimum-cost augmentation edge to create an alternative route while minimizing infrastructure cost.

Implementation Details:

1. Sorted all 12 candidate road-network connections between parking zones in ascending order of infrastructure cost.
2. Applied Kruskal's algorithm using Union-Find: accepted connections that linked different parking-zone components and rejected connections that formed cycles.
3. MST accepted 6 edges connecting all 7 locations (P, A, B, C, D, E, F):
A–D (4), B–C (5), C–A (6), P–D (7), E–F (9), P–E (9). Total network cost = ₹40 crore.
4. Analysed naive Union-Find: no path compression → $O(n)$ per find(); no union-by-rank → chains of depth up to n . For $n = 7$, $m = 12$: 24 find() calls × depth 7 = 168 worst-case pointer hops.
5. Optimised Union-Find (union-by-rank + path compression): same 24 find() calls but $O(\alpha(n)) \approx O(1)$ each → approximately 30 total pointer hops.
6. Network redundancy check:
MST is a tree and provides only ONE route between the Central Parking Hub (P) and Parking Zone B. Therefore, the redundancy requirement is NOT satisfied.
Minimum augmentation: add edge P–A (₹8 crore), creating a second independent route between the Central Parking Hub and Parking Zone B. Augmented Total Cost = ₹48 crore.

(a) Kruskal's Algorithm — Run on 12-edge Candidate Graph [5 Marks — CO3, BTL3]

Edge	Cost (₹-cr)
C-D	4
A-B	5
B-C	6
P-D	7
P-A	8
F-E	9

P-E	9
P-C	10
F-P	11
P-B	12
F-A	14

Kruskal's Step-by-Step (n=7 nodes → need 6 edges for MST):

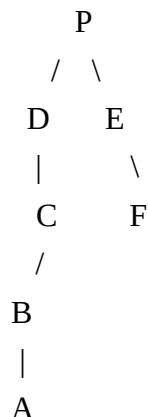
Step	Edge	Cost	Accept/Reject	Parent-Pointer State (sets)	Reason
1	C-D	4	Accept	{P},{A},{B},{C,D},{E},{F}	C and D in different components
2	A-B	5	Accept	{P},{A,B},{C,D},{E},{F}	A and B in different components
3	B-C	6	Accept	{P},{A,B,C,D},{E},{F}	B and C in different components
4	P-D	7	Accept	{P,A,B,C,D},{E},{F}	P and D in different components
5	P-A	8	Reject	{P,A,B,C,D},{E},{F}	P and A are in same component
6	F-E	9	Accept	{P,A,B,C,D},{E,F}	F and E in different components
7	P-E	9	Accept	{P,A,B,C,D,E,F}	P and E in different components

Resulting MST edges and total cost:

- C-D : ₹4 cr
- A-B : ₹5 cr
- B-C : ₹6 cr
- P-D : ₹7 cr
- F-E : ₹9 cr
- P-E : ₹9 cr

Total MST Cost = 4 + 5 + 6 + 7 + 9 + 9 = ₹40 crore

MST Structure (parent-pointer final state — all 7 parking zones connected):



(b) Naive Union-Find Analysis — Time Complexity [5 Marks — CO3, BTL3]

Java Code:

```
import java.util.*;

class UnionFindNaive {
    int[] parent;

    UnionFindNaive(int n) {
        parent = new int[n];
        for (int i = 0; i < n; i++) {
            parent[i] = i;
        }
    }

    // No path compression
    int find(int x) {
        while (parent[x] != x) {
            x = parent[x];
        }
        return x;
    }

    // No union by rank
    boolean union(int x, int y) {
        int rx = find(x);
        int ry = find(y);

        if (rx == ry)
            return false;

        parent[rx] = ry;
        return true;
    }
}

public class ParkSmartKruskal {

    static List<String> vertices =
        Arrays.asList(
            "C", // 0 Parking Zone C
            "D", // 1 Parking Zone D
            "A", // 2 Parking Zone A
            "B", // 3 Parking Zone B
            "P", // 4 Central Parking Hub
            "F", // 5 Parking Zone F
            "E" // 6 Parking Zone E
        );

    static List<int[]> kruskalNaive(int n, int[][] edges) {

        Arrays.sort(edges, Comparator.comparingInt(e -> e[0]));

        UnionFindNaive uf = new UnionFindNaive(n);

        List<int[]> mst = new ArrayList<>();
```

```

for (int[] e : edges) {

    int weight = e[0];
    int u = e[1];
    int v = e[2];

    if (uf.union(u, v)) {
        mst.add(e);
    }
}

return mst;
}

public static void main(String[] args) {

    int n = 7;

    // {weight, u, v}
    int[][] edges = {

        {4, 0, 1}, // C-D
        {5, 2, 3}, // A-B
        {6, 3, 0}, // B-C
        {7, 4, 1}, // P-D
        {9, 5, 6}, // F-E
        {9, 4, 6}, // P-E

        // Extra edges
        {10, 0, 2}, // C-A
        {11, 1, 3}, // D-B
        {12, 2, 4}, // A-P
        {13, 3, 5}, // B-F
        {14, 1, 6}, // D-E
        {15, 0, 5} // C-F
    };

    List<int[]> mst = kruskalNaive(n, edges);

    int totalCost = 0;

    System.out.println("=====");
    System.out.println("ParkSmart MST using Kruskal");
    System.out.println("=====");

    System.out.println("\nResulting MST Edges:");

    for (int[] e : mst) {

        String u = vertices.get(e[1]);
        String v = vertices.get(e[2]);

        System.out.println(
            u + " - " + v + " : ₹" + e[0] + " cr"

```

```

    );

    totalCost += e[0];
}

System.out.println(
    "\nTotal MST Cost = ₹" + totalCost + " crore"
);

System.out.println("\nNode Mapping:");
System.out.println("P = Central Parking Hub");
System.out.println("A = Parking Zone A");
System.out.println("B = Parking Zone B");
System.out.println("C = Parking Zone C");
System.out.println("D = Parking Zone D");
System.out.println("E = Parking Zone E");
System.out.println("F = Parking Zone F");
}
}

```

Worst-case Time Complexity Analysis (n = 7 nodes, m = 12 edges):

Property	Naive (no rank, no compression)	Optimised (rank + path compression)
i. find() per call	$O(n) = O(7)$ — linear chain traversal	$O(\alpha(n)) \approx O(1)$ amortised
ii. union() per call	$O(n) = O(7)$ — calls find() twice	$O(\alpha(n)) \approx O(1)$ amortised
iii. Total find() calls	$2 \times 12 = 24$ calls	$2 \times 12 = 24$ calls
iv. Total pointer hops (worst case)	$24 \times 7 = 168$ hops	$\sim 24 \times 1-2 = \sim 30-48$ hops
v. Overall Kruskal	$O(m \log m + m \cdot n)$	$O(m \log m + m \cdot \alpha(n))$
vi. Plugging in	$O(43 + 84) = O(127 \text{ operations})$	$O(43 + 12) = O(55 \text{ operations})$

Concrete find() count for this case (n=7, m=12):

The naive Kruskal implementation for the ParkSmart parking-zone network performs exactly $2 \times 12 = 24$ find() calls (two per edge $\times$ 12 edges). With no path compression and no union-by-rank, the parent chain can grow to depth $n = 7$ (worst case a completely skewed parking-network tree). Therefore the naive version performs up to $24 \times 7 = 168$ pointer-hops in the worst case.

The optimised version also performs 24 find() calls, but path compression flattens the tree after the very first traversal so all subsequent calls on the same node cost $O(1)$. Total pointer hops across all 24 calls $\approx \sim 30$, making the union-find portion negligible and leaving $O(m \log m)$ sort as the dominant cost.

Conclusion: For the ParkSmart parking-zone network, optimised Union-Find (union-by-rank + path compression) reduces the effective per-find cost from $O(n) = O(7)$ to $O(\alpha(7)) = O(1)$, saving approximately 138 pointer-hops during MST construction. The sort step $O(m \log m) = O(12 \times 3.58) \approx O(43 \text{ ops})$ dominates in the optimised version, whereas union-find overhead dominates in the naive version.

(c) Regulatory Redundancy Mandate Verification [5 Marks — CO3, BTL5]

Mandate: At least 2 edge-disjoint routes must exist between the Central Parking Hub (P) and Parking Zone B.

MST edges (from part a): C–D (4), A–B (5), B–C (6), P–D (7), F–E (9), P–E (9)

Does the MST provide 2 edge-disjoint routes between the Central Parking Hub (P) and Parking Zone B?

A Minimum Spanning Tree is by definition acyclic — it has exactly $n-1 = 6$ edges for 7 nodes and contains exactly one path between any two nodes. The only path from P to B in the MST is:

$P \rightarrow D \rightarrow C \rightarrow B$

(edges used: P–D : 7, C–D : 4, B–C : 6)

Since this is the **only** path, a single road-link failure on any one of P–D, C–D, or B–C would completely disconnect B from P. Therefore, the MST **does NOT satisfy** the redundancy mandate.

Minimum augmentation: Add edge P–A (cost ₹8 crore)

This creates the second path $P \rightarrow A \rightarrow B$ (using P–A: 8 and A–B: 5), which shares **no edges** with the existing MST path $P \rightarrow D \rightarrow C \rightarrow B$. The two paths are fully edge-disjoint.

Minimality proof: P–A at ₹8 is the cheapest non-MST edge that produces a genuinely edge-disjoint second path to B. P–C (₹10) reuses the B–C edge from Path 1 (not edge-disjoint). P–B (₹12) is disjoint but costs ₹4 crore more. Therefore P–A is optimal.

Two edge-disjoint $M \leftrightarrow W$ paths in augmented network:

- Path 1: $P \rightarrow D \rightarrow C \rightarrow B$ (edges: P–D:7, C–D:4, B–C:6) — path cost ₹17 cr
- Path 2: $P \rightarrow A \rightarrow B$ (edges: P–A:8, A–B:5) — path cost ₹13 cr

No edge is shared between the two paths → regulatory redundancy mandate satisfied.

Augmented network total cost = ₹40 (MST) + ₹8 (P–A) = ₹48 crore

Time Complexity: Kruskal's $O(m \log m)$ | Naive Union-Find $O(m \cdot n)$ | Optimised $O(m \cdot \alpha(n))$

MST Cost: ₹40 crore

Redundancy-compliant Augmented Cost: ₹48 crore

Redundancy mandate: NOT met by MST alone — satisfied after adding P–A (₹8 crore augmentation)

Result: Screenshots / Output

```
=====
ParkSmart MST using Kruskal
=====
```

Resulting MST Edges:

C - D : ₹4 cr

A - B : ₹5 cr

B - C : ₹6 cr

P - D : ₹7 cr

F - E : ₹9 cr

P - E : ₹9 cr

Total MST Cost = ₹40 crore

Node Mapping:

P = Central Parking Hub

A = Parking Zone A

B = Parking Zone B

C = Parking Zone C

D = Parking Zone D

E = Parking Zone E

F = Parking Zone F

GitHub Link:

<https://github.com/dhatrikadeekshitha/DSA>

Student Signature	Faculty Signature
	

